PORTFOLIO



책임감 있는 태도로 팀에 기여하고
최적의 해결책을 도출하는 백엔드 엔지니어 김승상

+82 010-9542-3258
rovin054@gmail.com
https://github.com/seungsang2000

INTRODUCTION

Careers

경력 사항

- **올포랜드 해양정보 활용시스템 유지관리팀 인턴**
  - 기간 : 2024.08.19 ~ 2024.12.31
  - 해양 SI팀에서 시스템 운영 지원 및 테스트 자동화 프로젝트 개발

교육 사항

- **삼성 청년 SW·AI 아카데미**
  - 기간 : 2025.07.08 ~ 2026.06.30, 1628시간
  - Java, Spring, Vue 등 웹 기술과 AI/모델링/파인튜닝 실습 이수
  - 4번의 팀 프로젝트에 참여

- **한성대학교 컴퓨터공학부**
  - 기간 : 2019.03.04 ~ 2025.02.21
  - 학점 : 4.38/4.5
  - 성적 우수 장학금 4회 수령

프로젝트 내역

- **SSAFY 자율 프로젝트 ‘CoMeet’**
  - 기간 : 2026.04 ~ 2026.05
  - 역할 : 백엔드 개발 및 LangGraph·RAG 기반 AI 기능 구현
- **삼성전자 VD사업부 프로젝트 ‘Panoptes’**
  - 기간 : 2026.02 ~ 2026.04
  - 역할 : 백엔드 개발 및 LangChain 기반 AI 기능 구현
- **SSAFY 공통 프로젝트 ‘Unblur’**
  - 기간 : 2026.01 ~ 2026.02
  - 역할 : 백엔드 개발 및 인프라(배포, CI/CD, 모니터링) 환경 구축
- **SSAFY 관통 프로젝트 ‘나우유씨집’**
  - 기간 : 2025.11 ~ 2025.12
  - 역할 : 백엔드 개발
- **ICT COC 피우다 프로젝트 ‘WearAgain’**
  - 기간 : 2025.09 ~ 2025.12
  - 역할 : 풀스택 개발 및 인프라(배포, CI/CD, 모니터링) 환경 구축
- **신한은행 해커톤 ‘HeyFy’**
  - 기간 : 2025.08 ~ 2025.08
  - 역할 : 백엔드 개발 및 배포 환경 구성 지원

INTRODUCTION

Certificate & Award

자격증

- **SQL Developer(SQLD)**
  - 취득일 : 2025.04.04
  - 발급기관 : 한국데이터산업진흥원
- **정보처리기사**
  - 취득일: 2025.06.13
  - 발급기관 : 한국산업인력공단

어학/외국어

- **OPIc**
  - 등급: **IH**
  - 외국어구분: 영어
  - 시험일: 2026.04.11

수상 내역

- **삼성전자 VD·네트워크사업부-SSAFY 연계 프로젝트 우수상**
  - 수상일자 : 2026.04.03
  - 주최기관 : 삼성전자
  - 삼성전자 VD사업부에서 이미지 기반 공정자동화 프로젝트 수행
- **제 15회 피우다 프로젝트 우수상**
  - 수상일자 : 2025.12.17
  - 주최기관 : 정보통신산업진흥원
  - 다시입다연구소와 협업하여 21%파티 디지털 전환 프로젝트 수행
- **SSAFY 1학기 프로젝트 최우수상**
  - 수상일자 : 2025.12.26
  - 주최기관 : 삼성전자
  - SSAFY의 1학기 관통 프로젝트에서 AI 기반 부동산 정보 분석 서비스 제작
  - SSAFY의 14기 대표로서 15기들에게 관통프로젝트 소개 및 노하우 공유

INTRODUCTION

Skills

Backend Skills



Java

- 객체지향 기반 백엔드 애플리케이션 개발
- 계층 분리 구조 설계 및 구현
- 컬렉션·스트림·제네릭 기반 로직 활용



Spring

- Spring Boot 기반 REST API 구현
- MVC 패턴 기반 서버 구조 설계
- JPA·Validation·Security 기반 기능 개발



SQL

- 테이블 구조 및 관계 설계
- 조인·집계 중심 데이터 조회 쿼리 작성
- N+1 및 동시성 이슈 대응 경험

Infra Skills



Nginx

- Reverse Proxy 기반 서버 구성
- 도메인·포트·라우팅 설정 관리
- 서비스별 요청 경로 분리 및 외부 접근 구조 정리



Monitoring

- Prometheus·Grafana·Loki·Promtail 기반 모니터링 구축
- 로그·지표 기반 문제 추적 및 원인 분석
- 운영 흐름 모니터링 및 장애 대응 분석



Docker

- 컨테이너 기반 실행 환경 구성
- Docker Compose 기반 서비스 환경 관리
- 볼륨·네트워크·포트 설정 및 배포 환경 구성

PROJECT

Unblur

소개 가치관 기반 블라인드 데이팅 플랫폼

기간 2026.01.06 ~ 2026.02.13

인원 6명

역할 백엔드 개발 / 인프라 구축

기술 스택

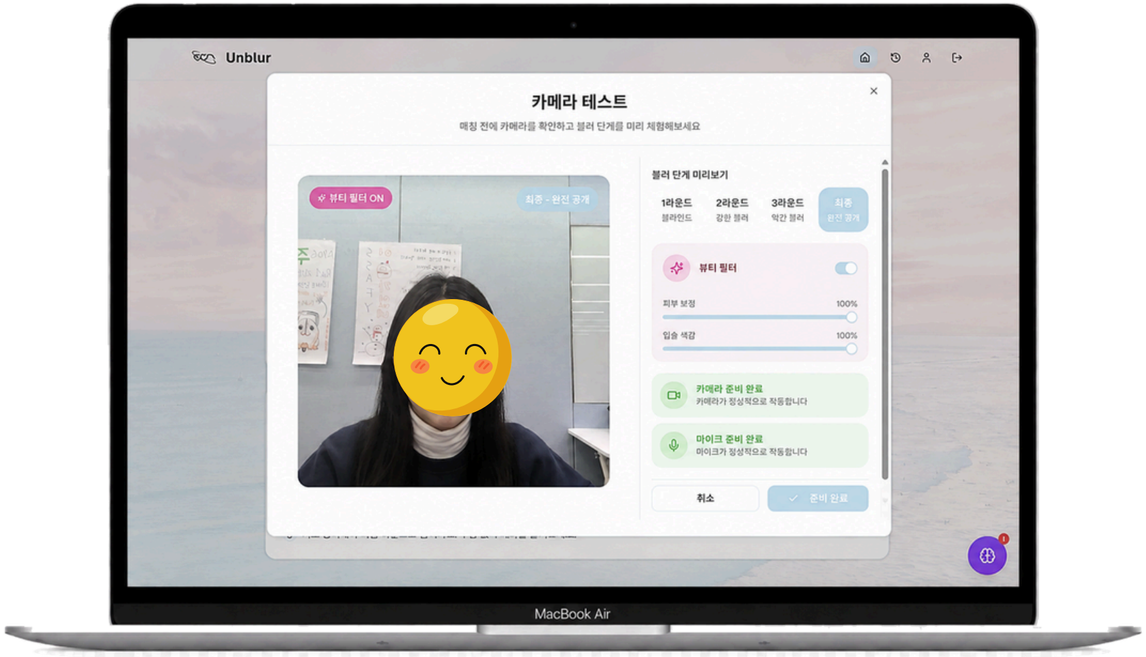
- 백엔드: Java, Spring Boot, PostgreSQL, SSE, STOMP, WebSocket
- 인프라/모니터링: Linux Server, Docker Compose, Nginx, MinIO, Kurento, Coturn, Prometheus, Grafana, Loki

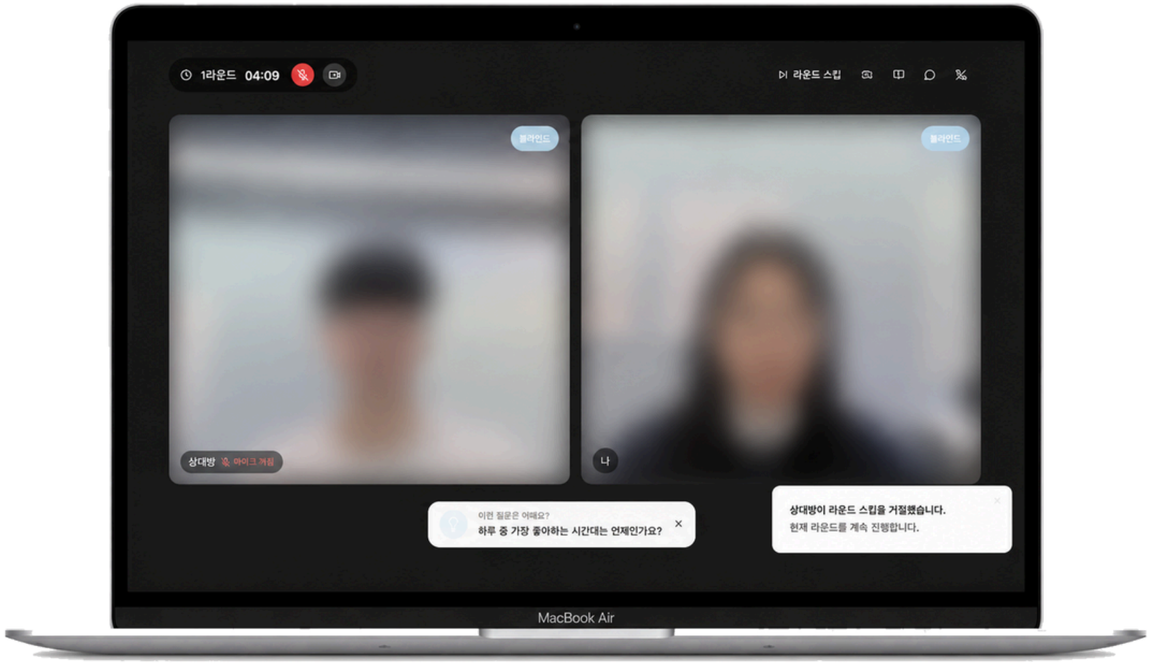
깃허브

- https://github.com/seungsang2000/Unblur

시연 영상

- https://youtu.be/Rjsn9n0uQd8





DETAIL

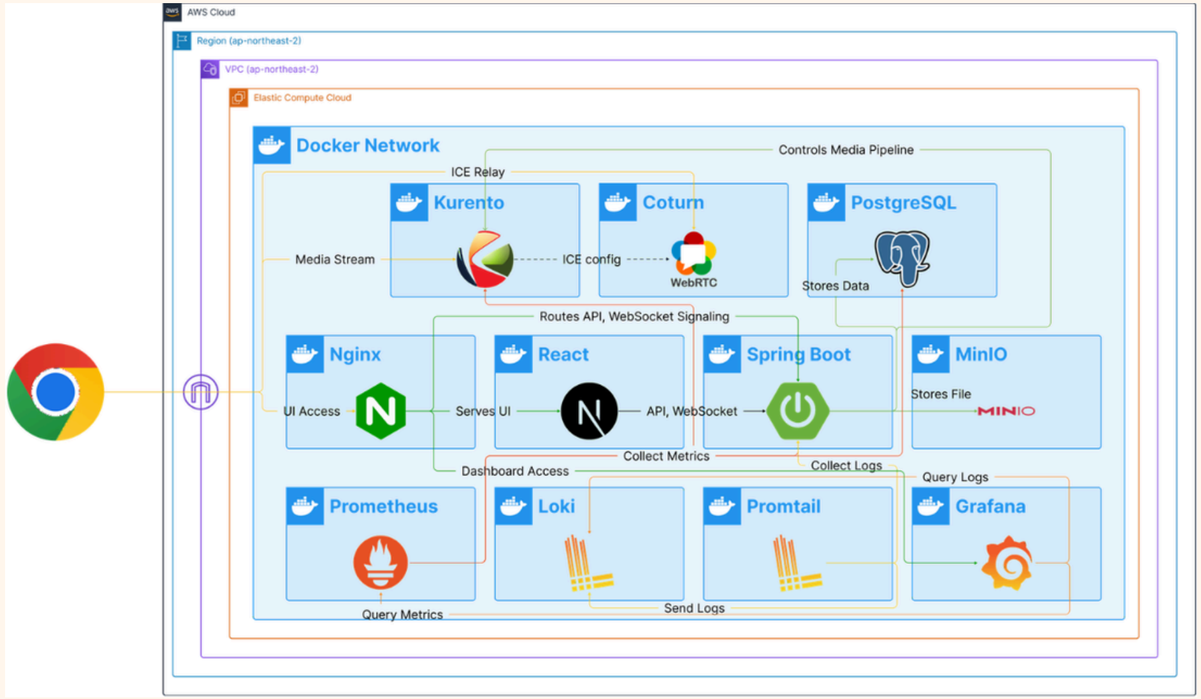
프로젝트 담당 역할

Backend

- REST 채팅 API와 STOMP Topic 구독 기반 실시간 브로드캐스트 구현
- WebSocket/Kurento 기반 WebRTC 시그널링 구현
- 라운드 시작·종료, 투표·스킵, 라운드 전환 이벤트 로직 구현
- Kurento 녹음 제어 및 MinIO 업로드 연동 구현
- MinIO 이벤트 기반 STT·AI 요약·DB 저장 비동기 파이프라인 구현

Infra

- Docker Compose 기반 Spring Boot, React, PostgreSQL, MinIO 운영 환경 구축
- Nginx 기반 API, WebSocket, SSE, MinIO, Grafana 라우팅 구성
- Kurento·Coturn WebRTC 인프라 및 Prometheus·Grafana·Loki 로그/모니터링 구성



Unblur - 아키텍처 구조도

TROUBLE SHOOTING

Coturn 기반 TURN Relay를 통한 WebRTC 연결 안정화

문제 상황

- 로컬에서는 연결됐지만 SSAFY 운영망에서는 WebRTC 연결 실패
- SSAFY 네트워크의 UDP 통신 제한으로 직접 미디어 경로 형성 불가
- 미디어 중계를 위한 TURN relay 경로 필요

목표 및 담당 역할

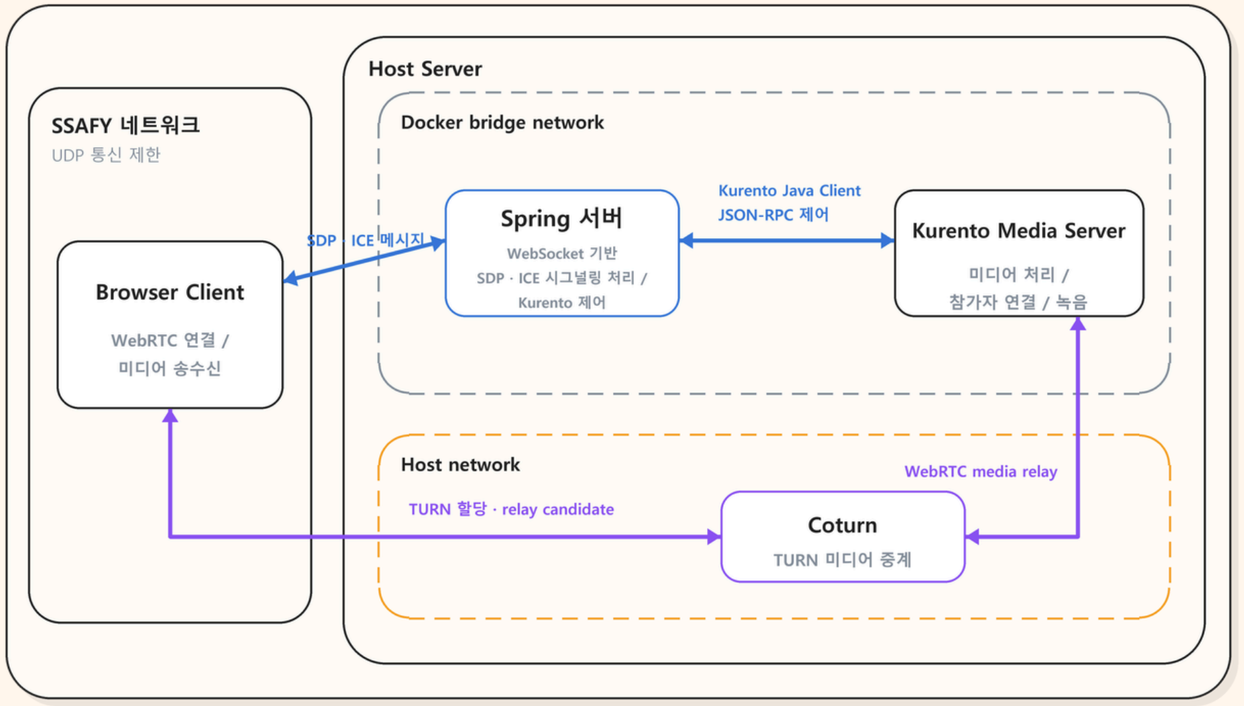
Spring WebSocket·Kurento 기반 WebRTC를 구축하고, Coturn을 연동해 방화벽 환경의 연결 안정성을 확보

실행 과정

1. Spring WebSocket 기반 시그널링과 Kurento 미디어 서버 연동 구현
2. SDP·ICE Candidate 전달 과정을 점검해 연결 실패 구간 추적
3. SSAFY 네트워크의 UDP 제한으로 직접 미디어 경로가 형성되지 않는 원인 확인
4. Coturn 구축 및 Kurento·클라이언트 ICE 설정 연동 후 TURN 경유 연결 검증

결과 및 성과

- Spring WebSocket·Kurento 기반 WebRTC 음성·영상 통신 기능 구축
- Coturn 기반 TURN relay로 방화벽 환경에서도 미디어 연결 경로 확보
- 시그널링부터 ICE Candidate 전달, TURN relay 구성까지 WebRTC 연결 전반의 구축·디버깅 경험 확보



해결 구조 - WebSocket 시그널링과 TURN 미디어 중계 구조

TROUBLE SHOOTING

Docker bridge 기반 대량 포트 처리 이슈

문제 상황

- Coturn을 Docker bridge network로 구성
- TURN relay를 위해 넓은 UDP 포트 범위를 ports로 노출
- 포트 매핑 과정에서 메모리 사용량이 급증하고 컨테이너 기동이 불안정해짐

목표 및 담당 역할

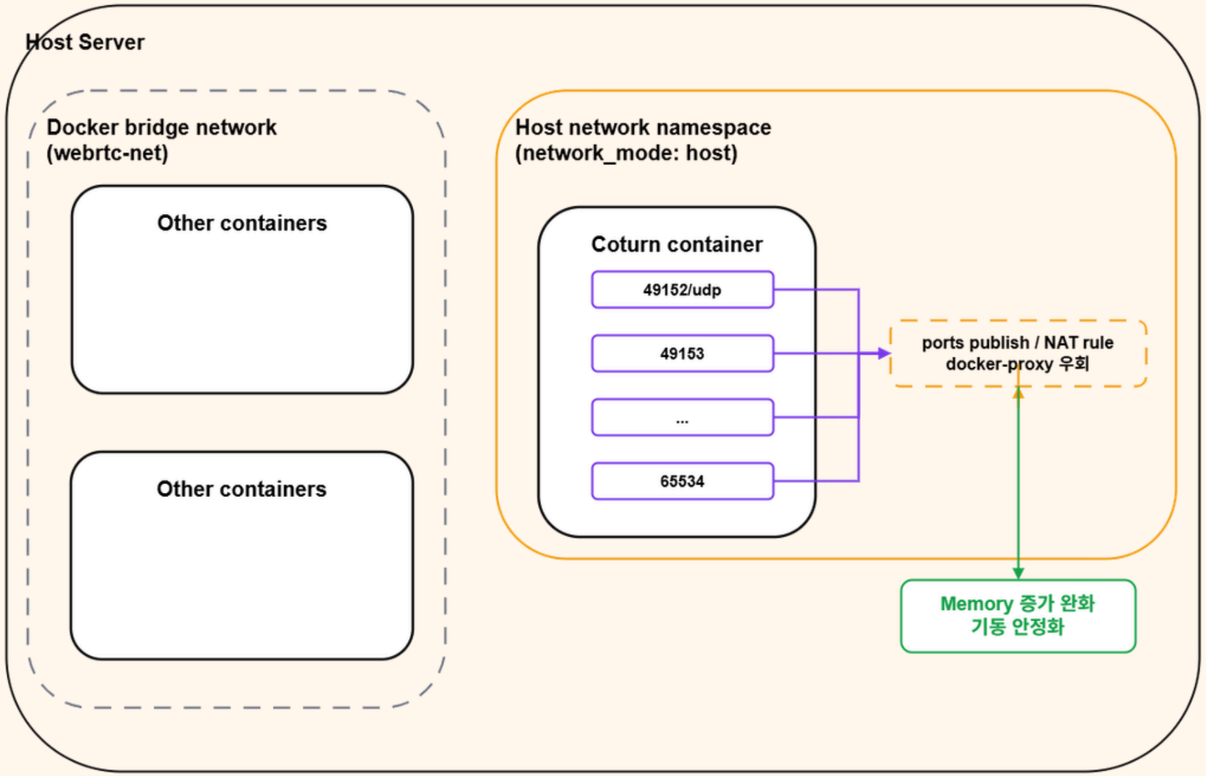
대량 UDP 포트 매핑에 따른 메모리 증가 원인을 분석하고, Coturn 네트워크 분리 및 기동 안정성 개선 담당

실행 과정

1. Swap 설정으로 서버 불안정을 임시 완화하고, 컨테이너를 순차 실행하며 리소스 증가 지점 추적
2. Coturn의 대량 UDP 포트 publish 과정에서 메모리 증가와 기동 불안정 발생 확인
3. 일반 서비스는 bridge network를 유지하고, Coturn만 network_mode: host로 분리해 동작 검증

결과 및 성과

- 대량 UDP 포트 매핑에 따른 메모리 증가 및 기동 불안정 완화
- Coturn 컨테이너와 WebRTC TURN 서버의 기동 안정성 개선
- Docker 네트워크 특성을 고려한 인프라 디버깅 경험 확보



해결 구조 - Coturn host network 분리

ACHIEVEMENTS

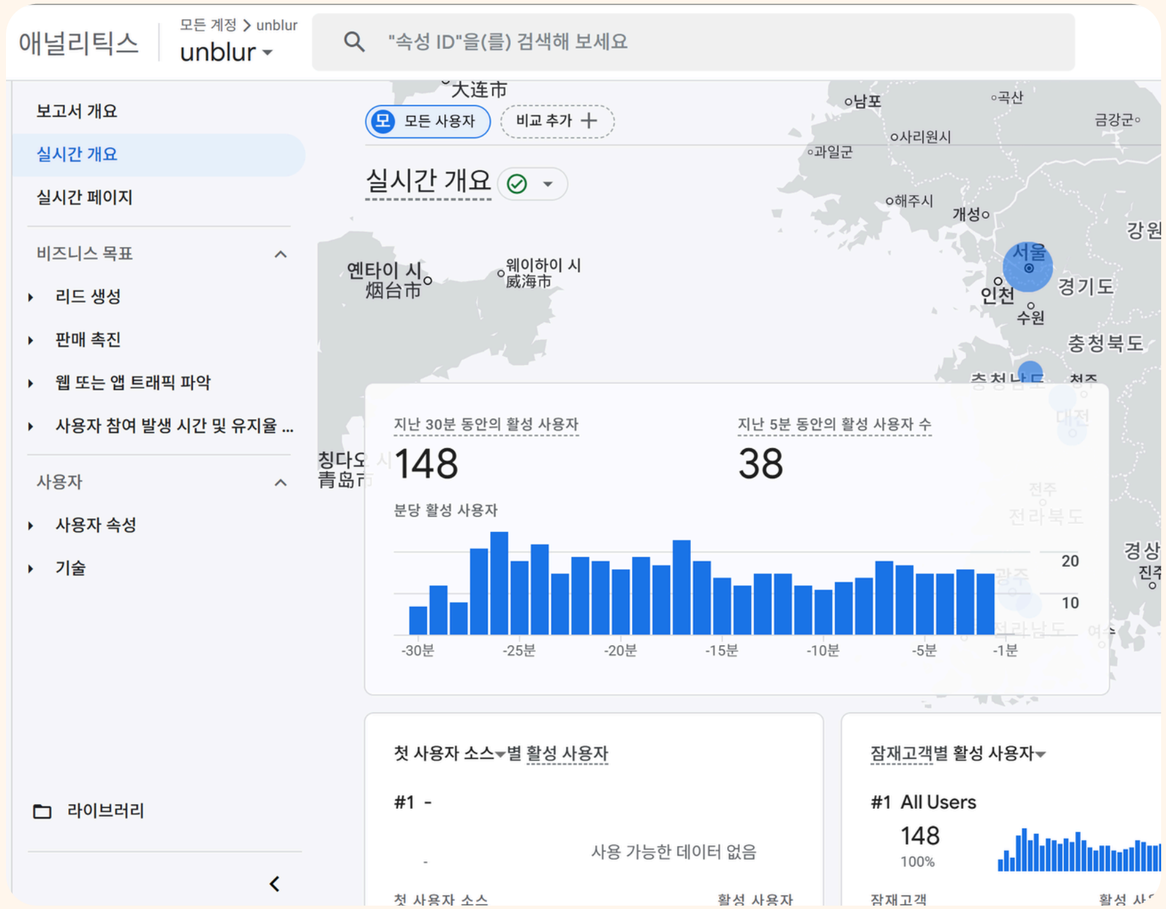
프로젝트 성과 및 회고

실제 사용자 트래픽 기반 운영 검증 경험

- Google Analytics 기준 **30분 동안 약 150명의 사용자가 유입**
- Swagger 우회 호출, 비정상 WebSocket 요청 등 예상하지 못한 사용자 행동 확인 및 대응
- Logback 기반 **일자별 운영 로그를 통해 인증 실패, SSE 연결 종료 등 주요 이슈 유형 분석 및 대응**

이벤트 기반 비동기 AI 후처리 구조 설계 경험

- 라운드 종료 시 Kurento 녹음 파일을 MinIO에 업로드하고, ObjectCreated webhook으로 후처리 이벤트 수신
- MinIO 이벤트 수신 API는 Internal Token으로 보호하고, STT·요약·DB 저장은 ThreadPoolTaskExecutor 기반 비동기 작업으로 분리
- 외부 AI API 호출 지연이 실시간 라운드 진행에 영향을 주지 않도록 WebSocket 이벤트 처리 흐름과 AI 후처리 파이프라인을 분리



Google Analytics - Unblur 관리 화면

PROJECT

WearAgain

소개 21%파티의 디지털 전환(DX)

기간 2025.09.18 ~ 2025.12.17

인원 5명

역할 풀스택 개발 / 인프라 구축

기술 스택

- 백엔드: Java, Spring Boot, Redis, MySQL, JWT, OAuth2
- 프론트엔드: React, TypeScript, Tailwind CSS, Zustand
- 인프라: Docker, Nginx, GitHub Actions, Prometheus, Loki, Grafana

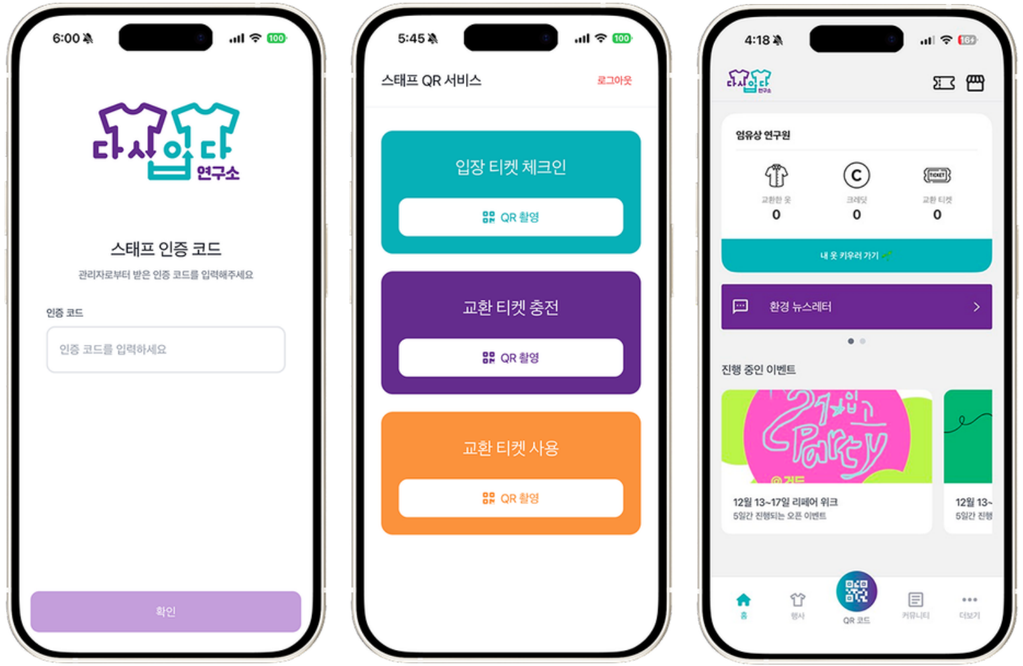
깃허브

- https://github.com/SSASINSA

시연 영상

- https://www.youtube.com/watch?v=J0CZWFB0iCk





DETAIL

프로젝트 담당 역할

Backend

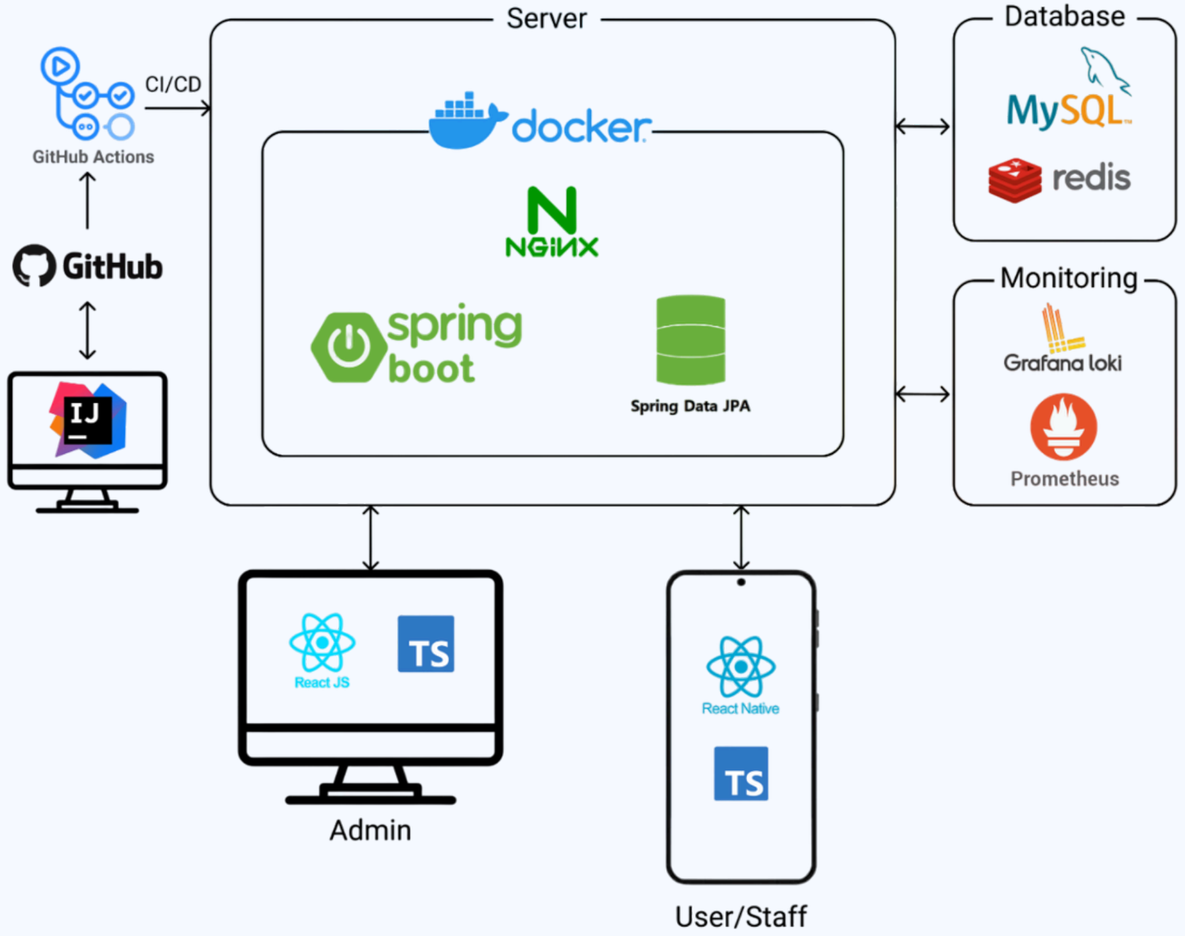
- 사용자 OAuth/JWT 설계
- 관리자 인증/인가 및 JWT/Redis 구조 구현
- 행사 참여 신청 및 운영 관리 API 구현
- Multipart 이미지 업로드 및 Nginx 정적 리소스 서빙 연계 구현
- Redis Lua 기반 재고·행사 정원 원자적 선점 및 DB 재동기화 구현

Frontend

- JWT 기반 관리자 인증 및 권한별 접근 제어 구현
- 행사, 상품, 주문, 게시글, 사용자/참가자 관리 화면 구현 및 연동

Infra

- GitHub Actions, Docker, Nginx 기반 CI/CD 및 배포 환경 구축
- MySQL, Redis, 환경변수, 배포 프로파일 기반 운영 환경 구성
- Prometheus, Grafana, Loki 기반 모니터링 환경 구축
- 배포 오류 대응 및 로그 기반 원인 분석



WearAgain - 아키텍처 구조도

TROUBLE SHOOTING

Redis 선점 카운터 기반 동시성 제어

문제 상황

- 여러 요청이 동일한 잔여 수량을 기준으로 처리되는 race condition으로 행사 정원과 굿즈 재고가 초과될 가능성 존재
- 초과 요청까지 모두 DB에 진입해 불필요한 DB 부하 발생

목표 및 담당 역할

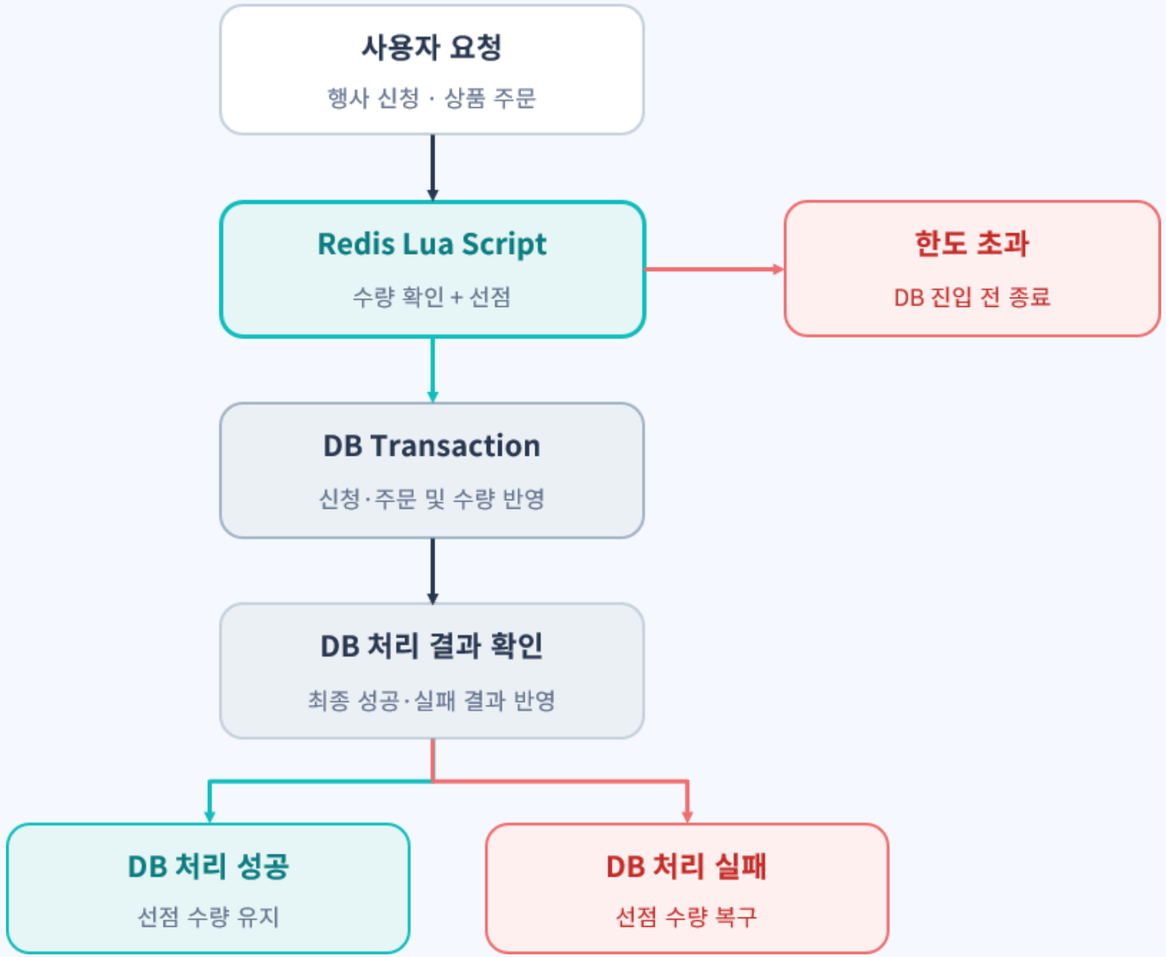
- Redis 선점 카운터로 초과 요청을 DB 진입 전에 차단하고 행사 정원·굿즈 재고·QR 처리의 정합성 개선

실행 과정

1. SERIALIZABLE과 조건부 원자 UPDATE 등 여러 방식을 검토하고, JUnit·JMeter를 활용해 정합성과 동시 요청 처리 성능을 비교·검증
2. 검증 결과를 바탕으로 Redis Lua Script 기반 원자적 선점 구조를 적용
3. 선점 결과를 구분해 성공한 요청만 DB로 전달하고, DB 처리 실패 시 Redis 수량 복구
4. QR 티켓은 GET과 DEL을 하나의 원자적 연산으로 처리해 중복 소비 차단

결과 및 성과

- 성공한 주문의 평균 응답 시간은 63.99%, 재고 부족으로 실패한 주문은 99.53% 단축
- 동시 요청에서도 행사 정원·굿즈 재고 초과를 방지하고 DB 실패 시 선점 복구
- QR 티켓의 중복 소비를 차단해 현장 처리 안정성 확보



TROUBLE SHOOTING

Read/Write Lock 기반 재동기화 제어

문제 상황

- 일반 요청(상품·행사 요청)의 Redis 수량 증감과 DB 기준 절대값 재동기화가 동시에 실행될 가능성 존재
- Scheduler·관리자 수정이 처리 중인 요청의 수량 변경을 덮어쓰는 lost update 발생 가능

목표 및 담당 역할

일반 요청의 동시 처리 성능을 유지하면서 같은 상품·행사 옵션의 재동기화 작업만 단독 실행되도록 경합 제어

실행 과정

1. 같은 상품·행사 옵션 단위로 하나의 Read/Write Lock을 공유
2. 일반 요청은 Redis 선점부터 DB 최종 처리 종료까지 Read Lock을 유지해 동시 실행 허용
3. 재동기화는 Write Lock을 사용해 진행 중인 모든 Read Lock 완료까지 대기
4. Write Lock 획득 후 DB 최신 수량을 다시 조회해 Redis에 절대값으로 반영

결과 및 성과

- 일반 요청은 병렬로 처리하고 재동기화 구간만 단독 실행해 처리량 유지
- 처리 중인 Redis 수량 변경이 DB 기준 절대값에 덮이는 문제 방지
- DB를 최종 기준으로 유지하면서 Redis와 DB 간 수량 정합성 보완



ACHIEVEMENTS

프로젝트 성과 및 회고

제 15회 피우다 프로젝트 우수상 수상

- **정보통신산업진흥원장상 수상**
- 63팀 중 2위
- 인터뷰와 요구사항 분석을 바탕으로 행사 신청, 의류 등록, 현장 운영, 관리자 검수 흐름을 구체화한 점에서 높은 평가를 받음

실제 기관과의 협업 기반 서비스 기획 경험

- 다시입다연구소와 소통하며 기존 행사 운영 방식과 문제 분석
- **요구사항이 명확히 정리되지 않은 상황**에서 인터뷰, 피드백 반영, 기능 우선순위 조정을 통해 **서비스를 구체화**

백엔드·인프라 연계 개발 경험

- Multipart 이미지 업로드와 Nginx 정적 리소스 서빙을 연계하여, 업로드 파일의 저장·조회 구조 설계
- Docker, Nginx, GitHub Actions 기반 배포 환경을 구축하고, 운영 서버 배포 및 장애 대응 흐름 총괄
- 개발 기능이 운영 환경에서 어떻게 동작하는지 고려하며, **배포·운영 관점에서 백엔드 구조를 설계하는 경험을 쌓음**



PROJECT

HeyFy

소개 외국인 유학생을 위한 금융 온보딩 서비스
(환율 정보 제공 및 예측 솔루션)

기간 2025.08.13 ~ 2025.08.31

인원 5명

역할 백엔드 개발 / 배포 환경 구성 지원

기술 스택

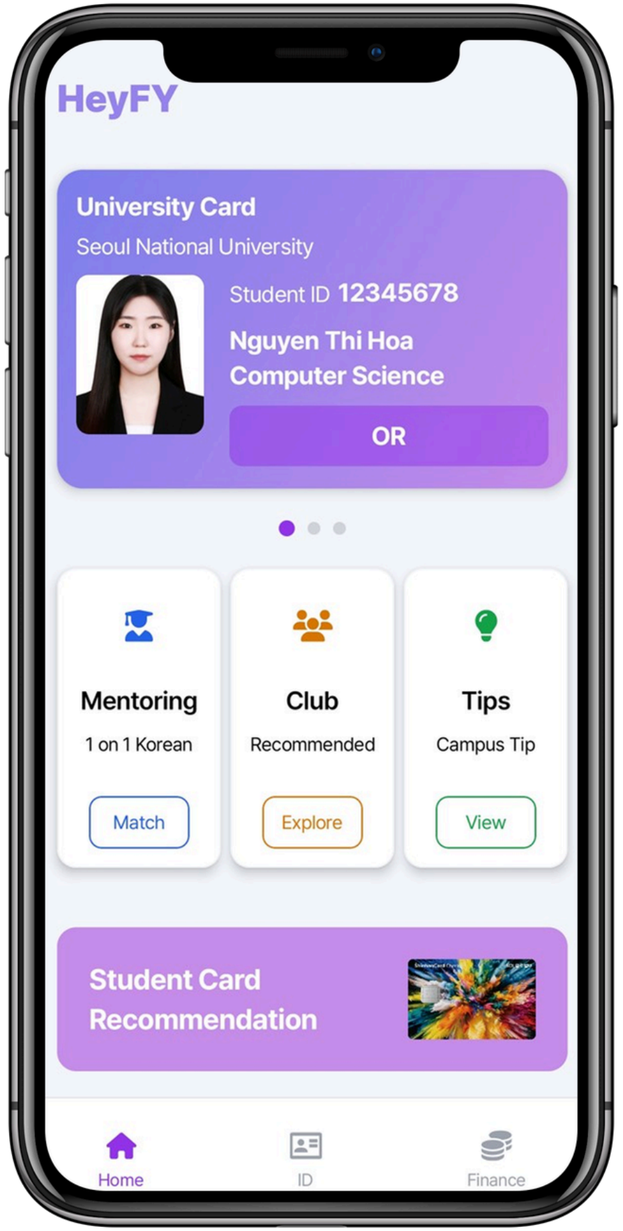
- 백엔드: Java, Spring Boot, JPA, Redis, MySQL, JWT
- 인프라 : Docker

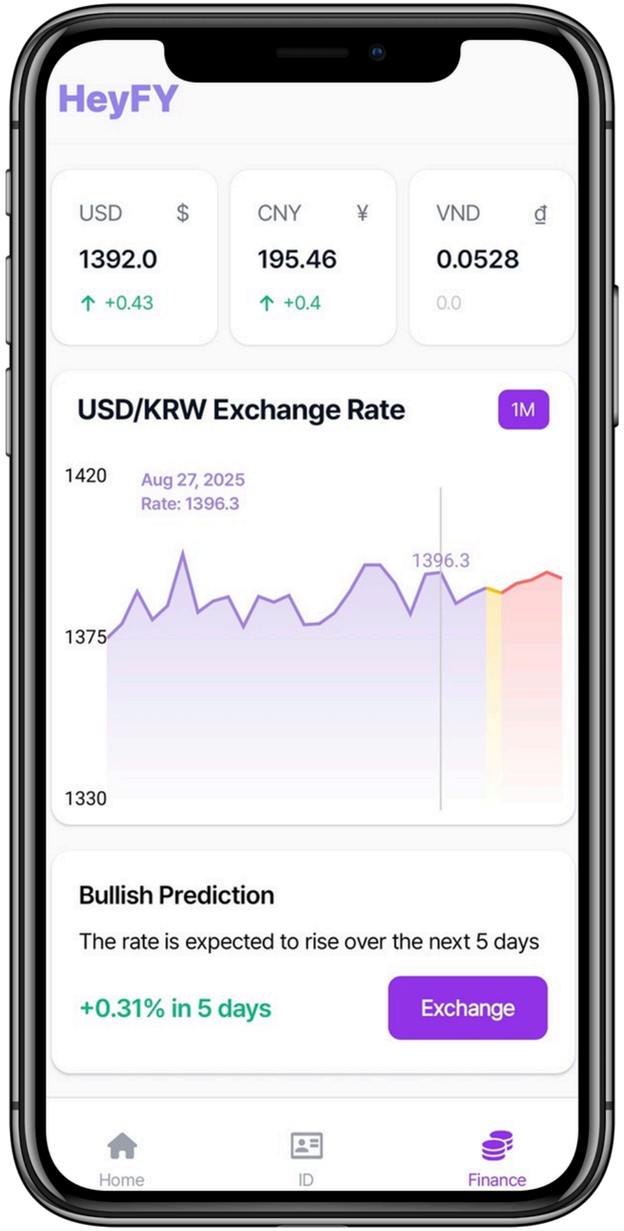
깃허브

- https://github.com/SSAFY-HeyFY

시연 영상

- https://youtu.be/cGSaggfDtPU





DETAIL

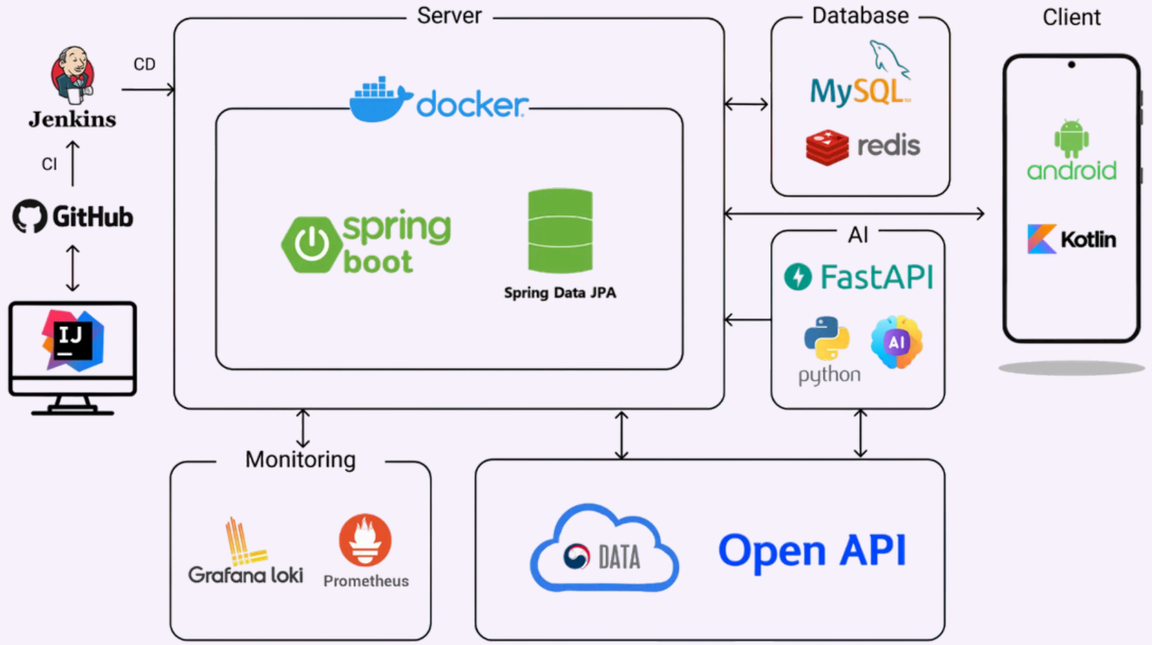
프로젝트 담당 역할

Backend

- JWT/RTR 기반 인증 구조 구현 및 AT·RT JTI 페어 검증 적용
- Redis 원자 연산 기반 JTI Lock으로 토큰 재발급 동시성 문제 해결
- Spring Security에 SID 검증 레이어를 추가해 민감 금융 API 접근 제어
- PIN 기반 거래 인증 공통화 및 Redis 실패 카운터·Lock 기반 거래 보호
- 신한은행 금융 API 기반 userKey 발급, 계좌 생성·등록, 1원 인증, 잔액·거래내역 조회 구현
- 신한은행 계좌·이체·환전 API를 조합해 원화↔외화 환전 거래 흐름 구현
- 신한 API 실패 코드를 잔액 부족, 계좌 오류, 환전 금액 오류 등 서비스 도메인 예외로 변환

Infra

- Dockerfile, Docker Compose 기반 백엔드 실행 환경 구성 및 관리 지원
- MySQL, Redis 컨테이너 기반 개발/배포 환경 운영 지원



HeyFy - 아키텍처 구조도

TROUBLE SHOOTING

JWT·SID 기반 금융 거래 보호 구조

문제 상황

- 금융 API에 대해 30분 단위 PIN 2차 인증 요구사항 발생
- 이미 구현된 RTR 기반 인증 구조를 전면 수정하기 어려워, 기존 흐름과 호환되는 추가 인증 레이어가 필요

목표 및 담당 역할

기존 RTR 기반 로그인 흐름을 유지하면서 SID 검증과 PIN 기반 2차 인증 구조를 추가하고, 관련 인증/인가 흐름 전 과정 설계 및 구현 담당

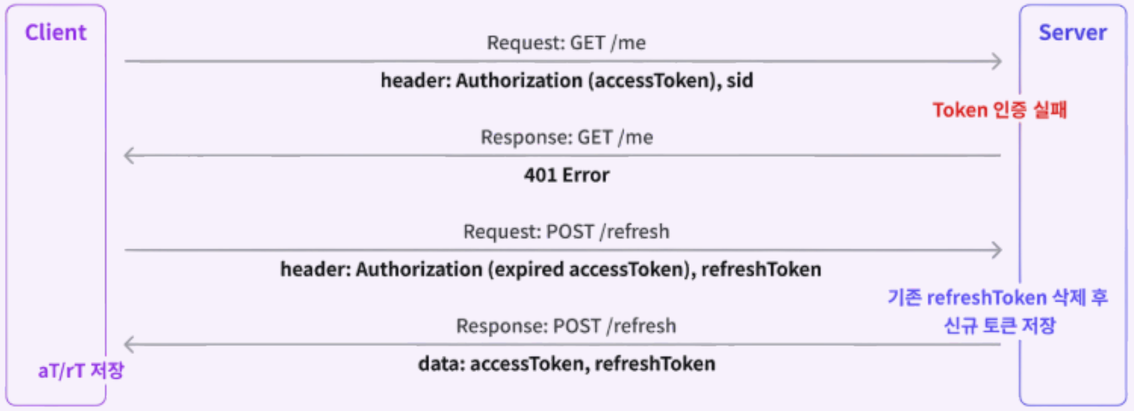
실행 과정

1. 기존 AT·RT 재발급 구조는 유지하고, 로그인 시 SID를 함께 발급하도록 확장
2. 금융 API 요청에 대해 JWT 인증 이후 SID 유효성을 추가 검증하는 필터 적용
3. PIN 인증 성공 시 기존 SID 무효화 후 신규 SID 재발급 로직 구현
4. SID 만료 시 PIN 재인증을 요구해 로그인 인증과 2차 인증 만료 주기 분리

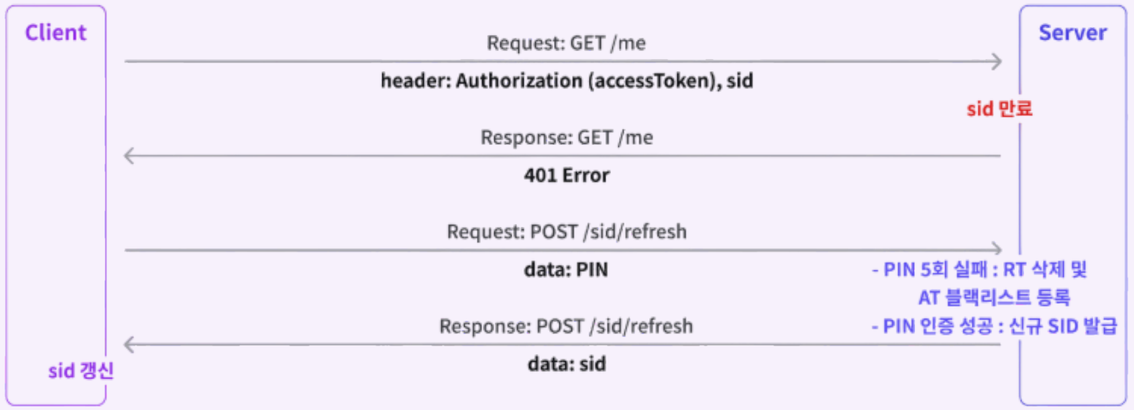
결과 및 성과

- 기존 RTR 구조를 유지하면서 금융 API 보호를 위한 2차 인증 레이어 구현
- AT·RT·SID 역할 분리를 통해 로그인 유지와 추가 인증 만료 주기를 독립적으로 제어
- 신한 본선 평가에서 신한 SOL과 유사한 인증 흐름이라는 긍정적인 피드백 받음

토큰 재발급 (AccessToken & RefreshToken)



세션 갱신 (SessionID)



ACHIEVEMENTS

프로젝트 성과 및 회고

신한 해커톤 본선 진출

- 예선 심사를 통과해 본선 발표 및 서비스 시연 진행
- 구현 결과물을 기반으로 기술 질의응답을 진행하며 백엔드 구조와 인증 흐름 설명
- 본선 과정에서 **신한 실무자와 기획, 서비스 흐름에 대한 피드백을 주고받으며 지속적으로 서비스를 개선**

제약 조건에 맞는 구조 설계 및 구현 경험

- 기존 JWT·RTR 인증 구조를 전면 개편하기 어려운 상황에서 PIN 기반 2차 인증 흐름 추가
- PIN 인증, SID 검증, 토큰 무효화를 조합해 금융 거래 보호 요구사항 반영
- **제한된 개발 기간 안에서 기존 구조와 호환되는 방식으로 기능을 확장**하고 결과물 완성

외부 금융 API 연동 경험

- 신한 금융 API를 활용해 계좌 조회/등록, 1원 인증, 이체, 환전 기능 구현
- 외부 API 응답 코드와 실패 케이스를 서비스 공통 ErrorResponse 구조로 변환
- 계좌, 이체, 환전 도메인별 예외 코드를 정의해 에러 응답 형식 통일
- Docker 기반 백엔드 실행 환경 구성 및 운영 지원

FINAL

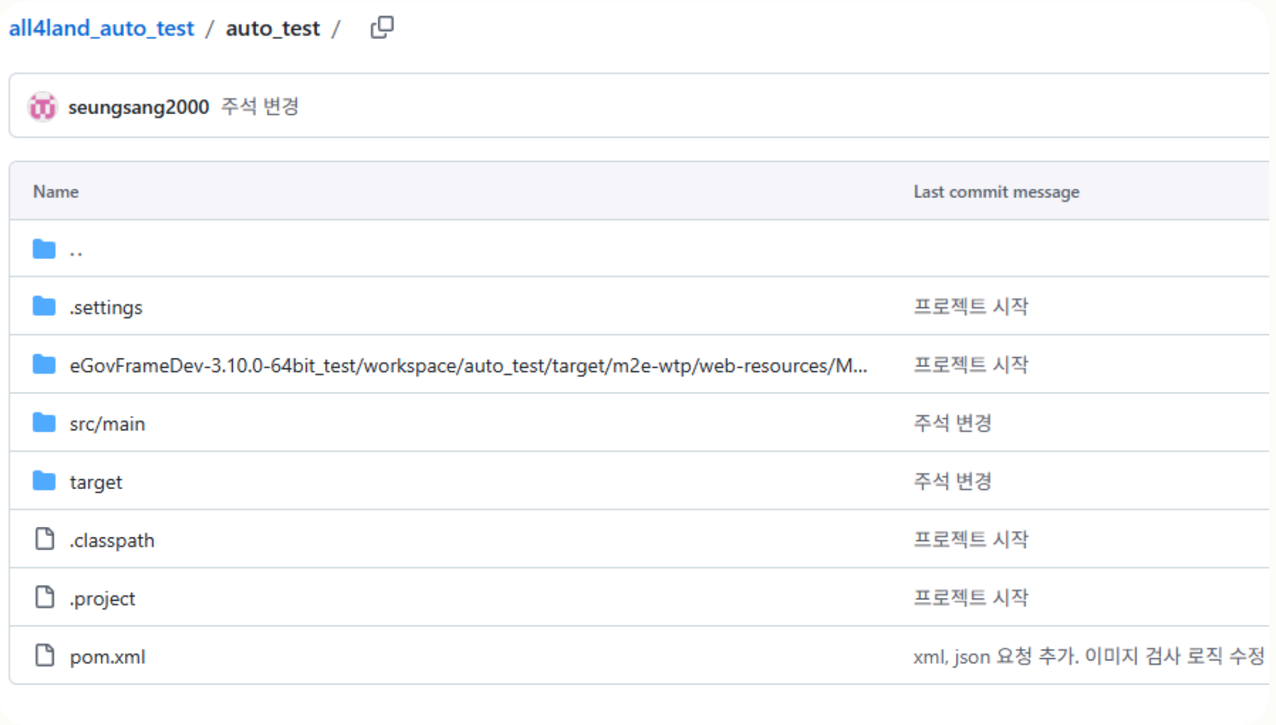
책임감 있는 태도로 팀에 기여하기 위하여 노력합니다.

저는 **제 기술과 지식을 통하여 팀에 기여하는 것**을
중요하게 생각합니다.

올포랜드에서 인턴십을 수행하면서 단순 업무 수행을 넘어
반복적인 테스트 업무의 비효율성을 개선하고자 테스트 도구를
직접 제안·제작하여 **팀의 업무 효율 향상에 기여**하였습니다.

단순 URL 호출로 진행되던 기존 테스트 방식을 엑셀
기반으로 표준화하고, 대상 프로젝트와의 관리 편의성을
고려해 전자정부프레임워크 기반으로 개발하였으며, **실무자
피드백을 지속적으로 반영**하면서 XML 응답을 효율적으로 처리하기
위해 SAX Parser 기반 이벤트 처리 방식을 적용하는 등의 작업을
거쳤습니다.

최종적으로 엑셀 업로드 → URL 테스트 수행 → 결과 엑셀
생성 구조의 테스트 자동화 도구를 구현하였습니다. 이를 통해
매주 3~4시간 소요되던 테스트 시간을 1시간 이하로 단축했습니다.



(GitHub) https://github.com/seungsang2000/all4land_auto_test

FINAL

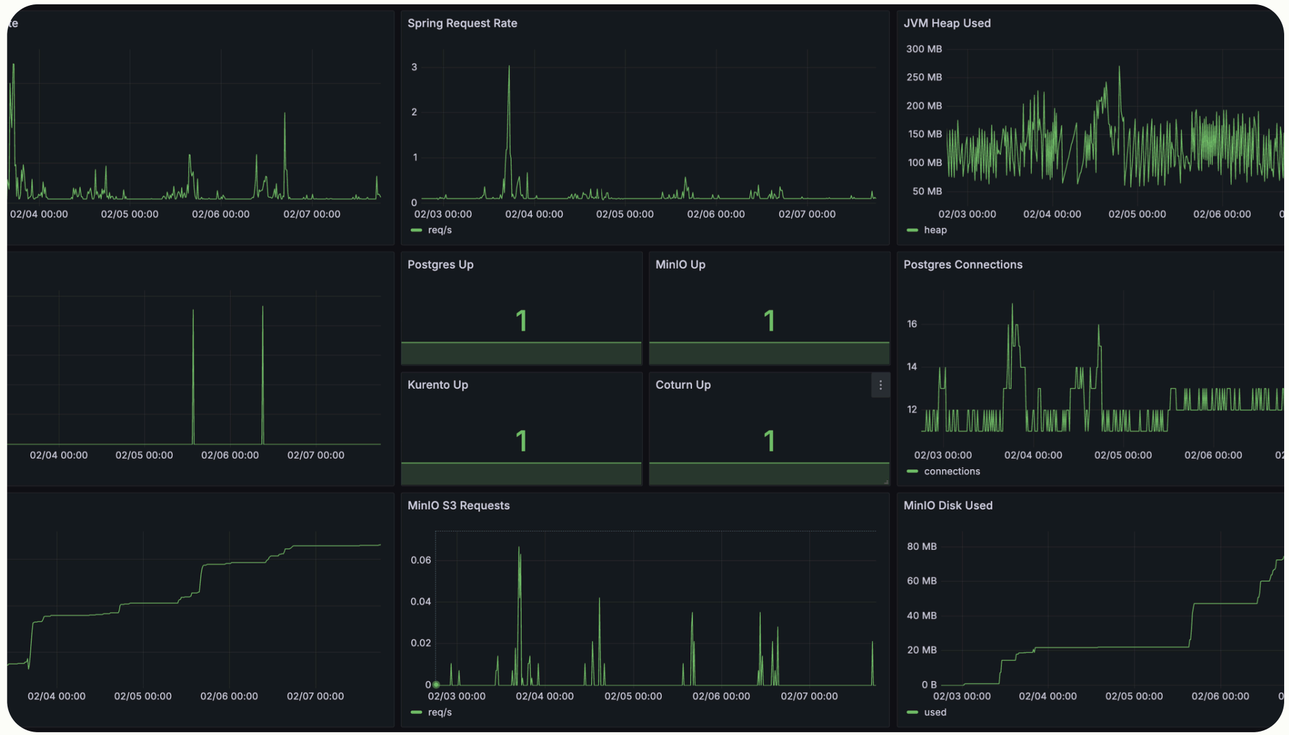
사전 준비를 통해 최적의 해결책을 도출합니다.

저는 최적의 해결책을 찾는 개발자로서, 문제를 해결하는 순간뿐 아니라 **그 해결책을 도출할 수 있도록 사전에 충분히 준비하는 과정을 중요하게 생각**합니다.

Unblur 프로젝트에서 Grafana·Loki·Prometheus 기반 **모니터링 환경과 일자별 로그 저장 구조를 선제적으로 구축**하고, 운영 중 수집된 WARN / ERROR 로그를 분석해 문제 원인을 추적했습니다.

이 과정에서 브라우저 권한 비활성화로 인한 WebSocket 세션 종료 문제를 포함한 **다양한 예외 상황을 확인**했고, 프론트엔드와 백엔드 구조를 함께 개선해 재발 가능성을 줄였습니다.

그 결과 재배포 시 ERROR 로그 비율을 **32.64%에서 2.69%로 감소**시키며 서비스 안정성을 높였습니다.



(Unblur 프로젝트) Grafana 대시보드를 통한 시스템 모니터링